EA3. Incorporando UI y BD:
Aplicación de Escritorio en Java Swing para Validar la Conexión a una Base de Datos con JDBC

David Esteban Zapata Rodríguez

Grupo 10

Institución Universitaria Digital de Antioquia

Ingeniería de Software y Datos

Programación Orientada a Objetos II

Docente: Jorge Armando Julio

2026

Tabla de Contenido

1. Introducción

2. Objetivos

3. Desarrollo

    1. Análisis de los Requisitos

    2. Diseño de la Solución con POO

    3. Interfaz Gráfica con Swing

    4. Conexión con JDBC y Cierre de Recursos

    5. Manejo de Excepciones

    6. Historial de Pruebas en CSV

    7. Evidencia de Ejecución

    8. Repositorio y Archivo ZIP

4. Conclusiones

5. Enlaces del Repositorio

6. Referencias

7. Anexo. Código Fuente Completo

Introducción

Casi todos los programas que usamos guardan su información en una base de datos: una tienda guarda sus productos, un banco sus cuentas y una app de transporte sus viajes. Antes de que un programa pueda leer o guardar algo, tiene que **conectarse** a esa base de datos, y para eso necesita saber dónde está (la URL), con qué usuario entrar y cuál es la contraseña. Si alguno de esos datos está mal, o si el servidor está apagado, la conexión falla, y el programa debe avisarle al usuario de forma clara en lugar de cerrarse con un error.

En Java, la conexión con bases de datos se hace con **JDBC** (*Java Database Connectivity*), una API que trae Java para hablar con cualquier motor de bases de datos usando el mismo código (Oracle, s.f.-a). Las ventanas, botones y cajas de texto se construyen con **Swing**, la librería gráfica que también viene incluida en Java (Oracle, s.f.-b).

En este trabajo se desarrolló **ConexionTester**, una aplicación de escritorio en Java con Swing, sin frameworks externos, donde el usuario escribe la configuración de una base de datos y, al presionar el botón "Probar conexión", el programa intenta conectarse. Si lo logra, muestra un mensaje verde de conexión exitosa; si falla, muestra un mensaje rojo con la explicación y el mensaje técnico del error. El informe explica cómo se construyó la interfaz, cómo se hace la conexión con `DriverManager` y *try-with-resources*, cómo se manejan las excepciones y cómo se organizó el código siguiendo los principios de la programación orientada a objetos.

Objetivos

Objetivo General

Desarrollar una aplicación de escritorio simple en Java, usando Swing, que permita al usuario ingresar la configuración de una base de datos y validar la conexión, manejando y mostrando cualquier error de JDBC que pueda ocurrir.

Objetivos Específicos

- Construir una interfaz gráfica con `JFrame`, `JPanel`, `JLabel`, `JTextField`, `JPasswordField`, `JButton` y `JTextArea`, organizada con los *layouts* `BorderLayout` y `GridLayout`.
- Programar el `ActionListener` del botón "Probar conexión" para que intente conectarse con `DriverManager.getConnection()` usando los datos escritos por el usuario.
- Manejar las excepciones `ClassNotFoundException` y `SQLException` con bloques `try-catch`, mostrando el resultado en verde o en rojo.
- Garantizar que la conexión siempre se cierre usando *try-with-resources*.
- Separar la lógica de conexión de la interfaz gráfica aplicando los principios de la programación orientada a objetos.

- Comprobar el funcionamiento con pruebas reales de conexión exitosa y fallida, y publicar el proyecto en GitHub.

Desarrollo

Análisis de los Requisitos

La Tabla 1 muestra cada requisito de la actividad y dónde se cumple en el código.

Tabla 1

Requisitos de la actividad y forma en que se cumplieron

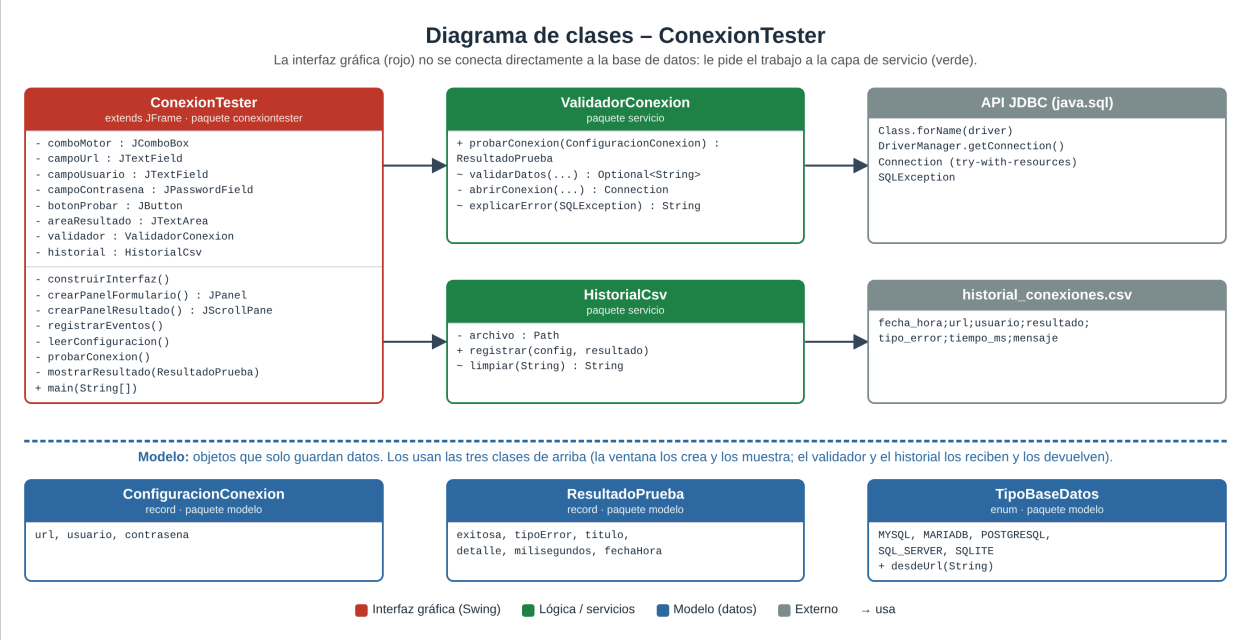
Requisito	Cómo se cumplió
Ventana principal (`JFrame`)	La clase `ConexionTester` extiende `JFrame`.
`JPanel` con `BorderLayout` o `GridLayout`	Panel principal con `BorderLayout`; etiquetas y campos en dos columnas con `GridLayout(4, 1)`.
`JLabel` + `JTextField` para la URL	`campoUrl`. Se llena con un ejemplo según el motor escogido.
`JLabel` + `JTextField` para el usuario	`campoUsuario`. Se desactiva con SQLite, que no usa usuario.
`JLabel` + `JPasswordField` para la contraseña	`campoContrasena`. Muestra asteriscos.
`JButton` “Probar conexión”	`botonProbar`. También se activa con la tecla Enter.
`JTextArea` dentro de un `JScrollPane`	`areaResultado`, de solo lectura, dentro de un `JScrollPane` con título.
`ActionListener` del botón	`botonProbar.addActionListener(evento -> probarConexion())`.
Conexión con `DriverManager`	`DriverManager.getConnection(url, usuario, contraseña)` en `ValidadorConexion`.
Capturar `ClassNotFoundException` y `SQLException`	Dos bloques `catch` en `ValidadorConexion.probarConexion`.
Mensaje verde de “conexión exitosa”	El texto se pinta de verde con `setForeground`.
Mensaje rojo de “fallo de conexión” + `e.getMessage()`	Texto rojo con el mensaje técnico, el SQLState y el código de error.
Cerrar la conexión siempre (*try-with-resources*)	`try (Connection conexion = abrirConexion(...))`.
Clase `ConexionTester` con arquitectura POO	`ConexionTester` es la ventana; la lógica está en clases separadas (Figura 1).

Diseño de la Solución con POO

La rúbrica pide que la lógica de conexión no quede mezclada con la interfaz. Por eso el proyecto se dividió en tres paquetes, cada uno con una responsabilidad (Figura 1). Así, si mañana la ventana se cambia por una página web, el `ValidadorConexion` se puede reutilizar sin cambiarle nada.

Figura 1

Diagrama de clases de la aplicación ConexionTester



Nota. Elaboración propia.

Tabla 2

Principios de la programación orientada a objetos aplicados

Principio	Dónde se aplicó
Encapsulamiento	Todos los componentes y atributos de las clases son `private`. Desde afuera solo se usan métodos públicos como `probarConexion` o `registrar`.
Responsabilidad única	`ConexionTester` solo maneja la ventana, `ValidadorConexion` solo se conecta, `HistorialCsv` solo escribe el archivo y los *records* solo guardan datos.
Herencia	`ConexionTester extends JFrame`: hereda todo lo que hace una ventana y le agrega sus propios componentes.
Abstracción	La ventana no sabe cómo se conecta JDBC: solo envía una `ConfiguracionConexion` y recibe un `ResultadoPrueba`.
Inyección de dependencias	El constructor de la ventana recibe el validador y el historial, en vez de crearlos adentro. Esto facilita cambiarlos o probarlos.
Nombres descriptivos (camelCase)	`campoContrasena`, `probarConexion()`, `mostrarResultado()`, `TipoBaseDatos`, etc.

Los datos viajan entre las clases en dos *records*. Un *record* es una clase cuyos valores no se pueden cambiar después de creada (OpenJDK, 2021). `ConfiguracionConexion` además cambia su `toString()` para que la contraseña nunca aparezca si alguien imprime el objeto:

```
/**
 * Datos que escribe el usuario en la ventana para conectarse.
```

```
 * Es un record: sus valores no se pueden cambiar despues de creado.
 *
 * @param url        URL JDBC, por ejemplo jdbc:mysql://localhost:3306/inventario
 * @param usuario    usuario de la base de datos (vacio en SQLite)
 * @param contrasena contrasena del usuario (vacia en SQLite)
 */
public record ConfiguracionConexion(String url, String usuario, String contrasena) {

    public ConfiguracionConexion {
        url = url == null ? "" : url.trim();
        usuario = usuario == null ? "" : usuario.trim();
        contrasena = contrasena == null ? "" : contrasena;
    }

    public boolean tieneUsuario() {
        return !usuario.isEmpty();
    }

    /** Evita que la contrasena aparezca si alguien imprime el objeto. */
    @Override
    public String toString() {
        return "ConfiguracionConexion[url=" + url + ", usuario=" + usuario + ", contrasena=****]";
    }
}
```

Interfaz Gráfica con Swing

La ventana usa un `BorderLayout`, que divide el espacio en cinco zonas (norte, sur, este, oeste y centro). En el norte va el título, en el centro el formulario y el resultado, y en el sur una barra de estado. Para que cada etiqueta quede alineada con su campo, el formulario usa dos columnas con `GridLayout`: las etiquetas en el oeste y los campos en el centro (Oracle, s.f.-c). Cada parte de la ventana se construye en su propio método, lo que hace el código más fácil de leer:

```
private void construirInterfaz() {
    setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);

    JPanel panelPrincipal = new JPanel(new BorderLayout(10, 10));
    panelPrincipal.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
    panelPrincipal.add(crearPanelEncabezado(), BorderLayout.NORTH);
    panelPrincipal.add(crearPanelCentral(), BorderLayout.CENTER);
    panelPrincipal.add(crearBarraEstado(), BorderLayout.SOUTH);

    setContentPane(panelPrincipal);
    pack();
    setMinimumSize(new Dimension(640, 520));
    setLocationRelativeTo(null);
}

/**
 * Formulario: una columna de etiquetas (WEST) y una de campos (CENTER),
 * cada una con GridLayout para que queden alineadas fila por fila.
 */
private JPanel crearPanelFormulario() {
    JPanel etiquetas = new JPanel(new GridLayout(4, 1, 5, 8));
    JPanel campos = new JPanel(new GridLayout(4, 1, 5, 8));

    agregarFila(etiquetas, campos, "Motor de base de datos:", comboMotor);
    agregarFila(etiquetas, campos, "URL de conexión:", campoUrl);
    agregarFila(etiquetas, campos, "Usuario:", campoUsuario);
    agregarFila(etiquetas, campos, "Contraseña:", campoContrasena);

    JPanel botones = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
    botonProbar.setFont(botonProbar.getFont().deriveFont(Font.BOLD));
    botones.add(botonLimpiar);
    botones.add(botonProbar);

    JPanel formulario = new JPanel(new BorderLayout(10, 10));
    formulario.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createTitledBorder("Configuración de la base de datos"),
            BorderFactory.createEmptyBorder(6, 8, 8, 8)));
    formulario.add(etiquetas, BorderLayout.WEST);
    formulario.add(campos, BorderLayout.CENTER);
    formulario.add(botones, BorderLayout.SOUTH);

    // Nombres para poder ubicar los componentes en las pruebas automaticas
    comboMotor.setName("comboMotor");
    campoUrl.setName("campoUrl");
    campoUsuario.setName("campoUsuario");
    campoContrasena.setName("campoContrasena");
    botonProbar.setName("botonProbar");
    areaResultado.setName("areaResultado");
    return formulario;
}
```

Además se agregó una lista desplegable (`JComboBox`) con los motores MySQL, MariaDB, PostgreSQL, SQL Server y SQLite. Al escoger uno, el campo URL se llena con un ejemplo, y si es SQLite se desactivan el

usuario y la contraseña, porque SQLite trabaja con un archivo y no los necesita.

El ActionListener del Botón

Un `ActionListener` es el código que se ejecuta cuando el usuario hace clic en el botón. Aquí se escribe con una *lambda*. El método `probarConexion()` lee los campos (nada está escrito fijo en el código), desactiva el botón y usa un `SwingWorker` para hacer la prueba en segundo plano. Sin esto, la ventana se "congelaría" mientras espera al servidor, que puede tardar hasta 5 segundos en responder si está apagado (Oracle, s.f.-d).

```
private void registrarEventos() {
    // ActionListener del boton "Probar conexion"
    botonProbar.addActionListener(evento -> probarConexion());
    botonLimpiar.addActionListener(evento -> limpiarResultado());
    comboMotor.addActionListener(evento -> seleccionarMotor((TipoBaseDatos)
comboMotor.getSelectedItem()));
    // Presionar Enter en cualquier campo tambien prueba la conexion
    getRootPane().setDefaultButton(botonProbar);
}

/** Lee los campos de texto y arma la configuracion (nada esta escrito fijo en el codigo). */
private ConfiguracionConexion leerConfiguracion() {
    return new ConfiguracionConexion(
            campoUrl.getText(),
            campoUsuario.getText(),
            new String(campoContrasena.getPassword()));
}

/**
 * Ejecuta la prueba en segundo plano con SwingWorker, para que la ventana
 * no se congele mientras espera al servidor.
 */
private void probarConexion() {
    ConfiguracionConexion configuracion = leerConfiguracion();
    botonProbar.setEnabled(false);
    setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));
    areaResultado.setForeground(Color.DARK_GRAY);
    areaResultado.setText("Probando conexión con " + configuracion.url() + " ...");

    new SwingWorker<ResultadoPrueba, Void>() {
        @Override
        protected ResultadoPrueba doInBackground() {
            return validador.probarConexion(configuracion);
        }

        @Override
        protected void done() {
            try {
                ResultadoPrueba resultado = get();
                mostrarResultado(resultado);
                guardarEnHistorial(configuracion, resultado);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } catch (ExecutionException e) {
                mostrarErrorInesperado(e.getCause());
            } finally {
                botonProbar.setEnabled(true);
                setCursor(Cursor.getDefaultCursor());
            }
        }
    }.execute();
}
```

Cuando llega el resultado, `mostrarResultado` escoge el color según el tipo de resultado: verde si fue exitosa, rojo si falló la conexión o el driver, y naranja si faltan datos:

```
/** Verde si la conexion funciono, rojo si fallo, naranja si faltan datos. */
private void mostrarResultado(ResultadoPrueba resultado) {
    Color color = switch (resultado.tipoError()) {
        case NINGUNO -> VERDE;
        case DATOS_INVALIDOS -> NARANJA;
        case DRIVER_NO_ENCONTRADO, FALLO_CONEXION -> ROJO;
    };
    areaResultado.setForeground(color);
    areaResultado.setText("[" + resultado.fechaHora().format(FORMATO_HORA) + "] "
            + resultado.titulo() + "  (" + resultado.milisegundos() + " ms)\n\n"
            + resultado.detalle());
    areaResultado.setCaretPosition(0);
}
```

Conexión con JDBC y Cierre de Recursos

Toda la lógica de conexión está en el método `probarConexion` de la clase `ValidadorConexion`, que no tiene nada de Swing. El método sigue tres pasos:

1. **Revisar los datos:** si la URL está vacía o no empieza por `jdbc:`, avisa sin intentar conectarse.
2. **Cargar el driver** con `Class.forName(...)`. El driver es el "traductor" entre Java y cada motor de base de datos, y viene en un archivo `.jar`. Según el inicio de la URL, el enum `TipoBaseDatos` sabe qué driver cargar; por ejemplo, para `jdbc:mysql:` carga `com.mysql.cj.jdbc.Driver`. Si el `.jar` no está, Java lanza `ClassNotFoundException`.
3. **Abrir la conexión** con `DriverManager.getConnection(url, usuario, contraseña)` dentro de un *try-with-resources*.

```
/**
 * Intenta abrir una conexion con los datos recibidos y la cierra enseguida.
 *
 * @param configuracion URL, usuario y contrasena escritos por el usuario
 * @return el resultado de la prueba (exito o el tipo de error con su mensaje)
 */
public ResultadoPrueba probarConexion(ConfiguracionConexion configuracion) {
    long inicio = System.currentTimeMillis();

    // 1. Revisar que los datos tengan sentido antes de intentar conectar
    Optional<String> errorDatos = validarDatos(configuracion);
    if (errorDatos.isPresent()) {
        return ResultadoPrueba.error(TipoError.DATOS_INVALIDOS, "DATOS INCOMPLETOS",
                errorDatos.get(), calcularTiempo(inicio));
    }

    Optional<TipoBaseDatos> tipo = TipoBaseDatos.desdeUrl(configuracion.url());

    try {
        // 2. Cargar el driver JDBC. Si el .jar no esta en la carpeta lib,
        //    Java lanza ClassNotFoundException.
        if (tipo.isPresent()) {
            Class.forName(tipo.get().getClaseDriver());
        }

        // 3. Abrir la conexion con try-with-resources: Java la cierra
        //    automaticamente al salir del bloque, salga bien o salga mal.
        DriverManager.setLoginTimeout(TIEMPO_ESPERA_SEGUNDOS);
        try (Connection conexion = abrirConexion(configuracion)) {
            String detalle = describirConexion(conexion, tipo);
            return ResultadoPrueba.exito(detalle, calcularTiempo(inicio));
        }

    } catch (ClassNotFoundException e) {
        String motor = tipo.map(TipoBaseDatos::getNombre).orElse("desconocido");
        String detalle = "No se encontró el driver JDBC de " + motor + ".\n"
                + "Copie el archivo .jar del driver en la carpeta 'lib' del proyecto y vuelva a ejecutar
el programa.\n\n"
                + "Mensaje técnico: " + e.getMessage();
        return ResultadoPrueba.error(TipoError.DRIVER_NO_ENCONTRADO, "FALLO DE CONEXIÓN: DRIVER NO
ENCONTRADO",
                detalle, calcularTiempo(inicio));

    } catch (SQLException e) {
        String detalle = explicarError(e) + "\n\n"
                + "Mensaje técnico: " + e.getMessage() + "\n"
                + "SQLState: " + e.getSQLState() + " | Código de error: " + e.getErrorCode();
        return ResultadoPrueba.error(TipoError.FALLO_CONEXION, "FALLO DE CONEXIÓN",
```

```
                detalle, calcularTiempo(inicio));
    }
}
```

¿Qué es try-with-resources? Es una forma de escribir el `try` en la que el recurso, en este caso la conexión, se declara entre paréntesis: `try (Connection conexion = ...)`. Al salir del bloque, ya sea porque todo salió bien o porque ocurrió un error, Java llama automáticamente a `conexion.close()` (Oracle, s.f.-e). Es como una puerta que se cierra sola: no hay que acordarse de escribir un `finally`, y nunca quedan conexiones abiertas ocupando el servidor. Si la conexión ni siquiera se logra abrir, no queda nada que cerrar.

Cuando la conexión funciona, el programa le pregunta a la base de datos su nombre, su versión y el driver usado con `DatabaseMetaData`, y comprueba que la conexión esté viva con `isValid()`. Como SQLite no usa usuario, si el campo está vacío se llama a `getConnection(url)` solo con la URL:

```
/** SQLite no usa usuario: en ese caso se llama a getConnection solo con la URL. */
private Connection abrirConexion(ConfiguracionConexion configuracion) throws SQLException {
    if (configuracion.tieneUsuario()) {
        return DriverManager.getConnection(configuracion.url(), configuracion.usuario(),
                configuracion.contrasena());
    }
    return DriverManager.getConnection(configuracion.url());
}
```

Manejo de Excepciones

Una *excepción* es la forma en que Java avisa que algo salió mal. Si no se atrapa con `try-catch`, el programa se cierra de golpe. Aquí se atrapan las dos excepciones que pide la actividad y se convierten en un mensaje claro para el usuario, sin perder el mensaje técnico (`e.getMessage()`), que le sirve al programador.

Tabla 3

Errores que maneja la aplicación

Situación	Excepción	Mensaje que ve el usuario	Color
URL vacía o sin `jdbc:`	(se revisa antes de conectar)	DATOS INCOMPLETOS + ejemplo de URL	Naranja
Falta el `.jar` del driver	`ClassNotFoundException`	FALLO DE CONEXIÓN: DRIVER NO ENCONTRADO	Rojo
Usuario o contraseña incorrectos	`SQLException` (SQLState 28000, código 1045)	El usuario o la contraseña son incorrectos	Rojo
La base de datos no existe	`SQLException` (código 1049)	El servidor respondió, pero la base de datos no existe	Rojo
Usuario sin permiso sobre la base	`SQLException` (código 1044)	El usuario no tiene permiso sobre esa base de datos	Rojo
Servidor apagado o puerto equivocado	`SQLException` (SQLState 08xxx)	No se pudo llegar al servidor	Rojo
URL que ningún driver entiende	`SQLException` (“No suitable driver”)	La URL no corresponde a ningún driver	Rojo
Conexión correcta	—	CONEXIÓN EXITOSA + datos del servidor	Verde

Nota. Los códigos 1044, 1045 y 1049 son los que usan MySQL y MariaDB.

Para elegir la explicación se usa el *SQLState*, un código estándar de cinco caracteres que dice qué tipo de error ocurrió (por ejemplo, los que empiezan por “28” son de autorización y los que empiezan por “08” son de conexión), junto con el código de error propio del motor (Oracle, s.f.-f):

```
/** Traduce los errores mas comunes a un mensaje facil de entender. */
String explicarError(SQLException e) {
    String estado = e.getSQLState() == null ? "" : e.getSQLState();
    String mensaje = e.getMessage() == null ? "" : e.getMessage().toLowerCase();

    if (mensaje.contains("no suitable driver")) {
        return "La URL no corresponde a ningún driver cargado. Revise que esté bien escrita.";
    }
    if (e.getErrorCode() == 1049 || mensaje.contains("unknown database")) {
        return "El servidor respondió, pero la base de datos no existe.";
    }
    if (e.getErrorCode() == 1044) {
        return "El usuario y la contraseña son correctos, pero ese usuario no tiene permiso "
                + "sobre esa base de datos (o la base de datos no existe).";
    }
    if (estado.startsWith("28") || e.getErrorCode() == 1045 || mensaje.contains("access denied")) {
        return "El usuario o la contraseña son incorrectos.";
    }
    if (estado.startsWith("08") || mensaje.contains("communications link failure")
```

```
            || mensaje.contains("connection refused") || mensaje.contains("could not connect")) {
        return "No se pudo llegar al servidor. Revise que esté encendido y que el host y el puerto sean
correctos.";
    }
    return "La base de datos rechazó la conexión.";
}
```

Historial de Pruebas en CSV

Como extra, cada prueba se guarda en el archivo `historial_conexiones.csv`, separado por punto y coma, para poder abrirlo en Excel. Se guardan la fecha y hora, la URL, el usuario, el resultado, el tipo de error, el tiempo en milisegundos y el mensaje. **La contraseña nunca se guarda**, por seguridad. El método `limpiar` pone entre comillas los textos que traen punto y coma, para no dañar las columnas del archivo.

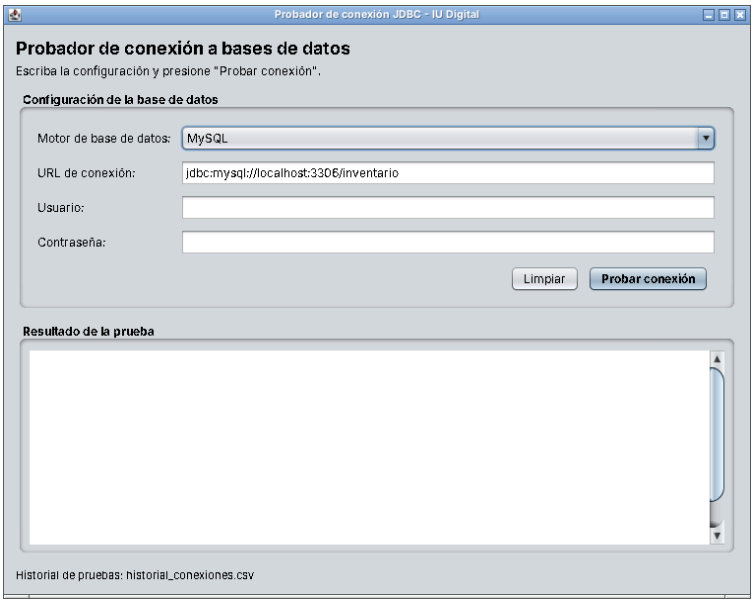
```
/**
 * Agrega una fila al CSV. Si el archivo no existe, primero escribe el encabezado.
 *
 * @throws IOException si no se puede escribir el archivo
 */
public void registrar(ConfiguracionConexion configuracion, ResultadoPrueba resultado) throws IOException {
    if (Files.notExists(archivo)) {
        // El BOM al inicio hace que Excel muestre bien las tildes
        Files.writeString(archivo, "" + ENCABEZADO + System.lineSeparator(), StandardCharsets.UTF_8);
    }
    String fila = String.join(";",
            resultado.fechaHora().format(FORMATO_FECHA),
            limpiar(configuracion.url()),
            limpiar(configuracion.usuario()),
            resultado.exitosa() ? "EXITOSA" : "FALLIDA",
            resultado.tipoError().name(),
            String.valueOf(resultado.milisegundos()),
            limpiar(resultado.detalle().replace("\n\n", "\n").replace("\n", " | ")));
    Files.write(archivo, List.of(fila), StandardCharsets.UTF_8, StandardOpenOption.APPEND);
}
```

Evidencia de Ejecución

La aplicación se compiló con Java 21 y se probó en ocho situaciones distintas: tres conexiones exitosas o de validación y cinco errores reales. Para las pruebas con servidor se usó **MariaDB 10.11**, un motor compatible con MySQL que maneja los mismos códigos de error, y para la prueba de archivo se usó **SQLite**. Las Figuras 2 a 9 muestran los resultados.

Figura 2

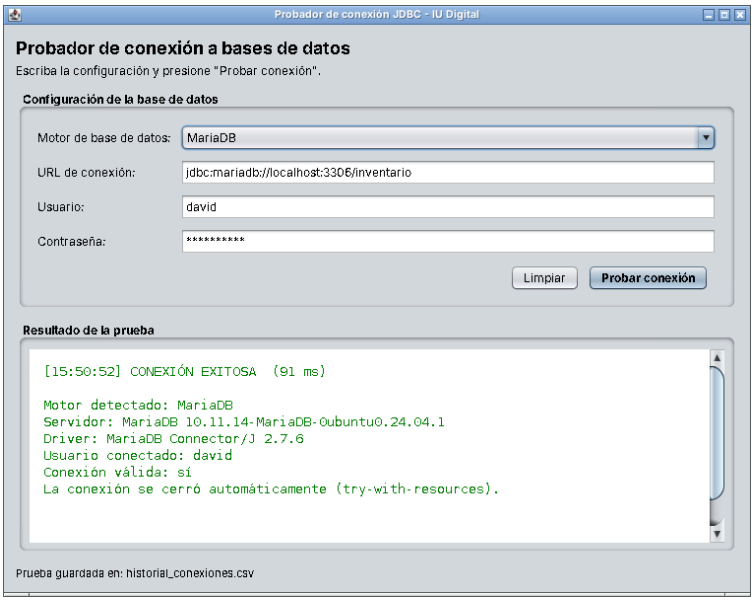
Ventana al abrir la aplicación



Nota. La URL de ejemplo de MySQL se llena sola. Captura de la aplicación en ejecución.

Figura 3

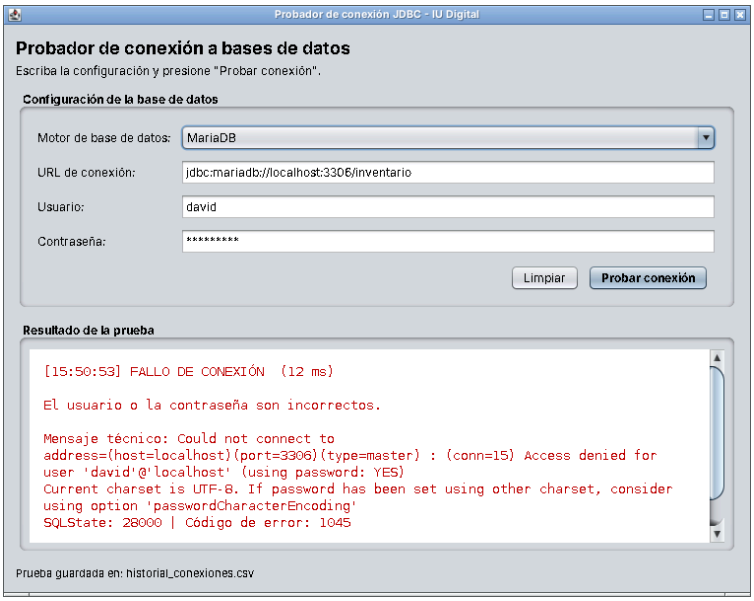
Conexión exitosa (mensaje verde)



Nota. Conexión a un servidor MariaDB 10.11 con el usuario y la contraseña correctos. Captura de la aplicación en ejecución.

Figura 4

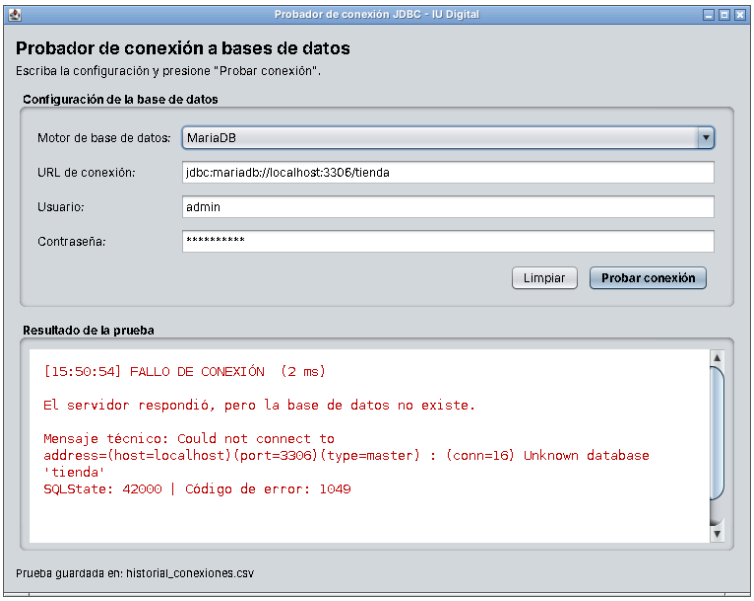
Contraseña incorrecta (SQLException)



Nota. Error 1045, SQLState 28000. Captura de la aplicación en ejecución.

Figura 5

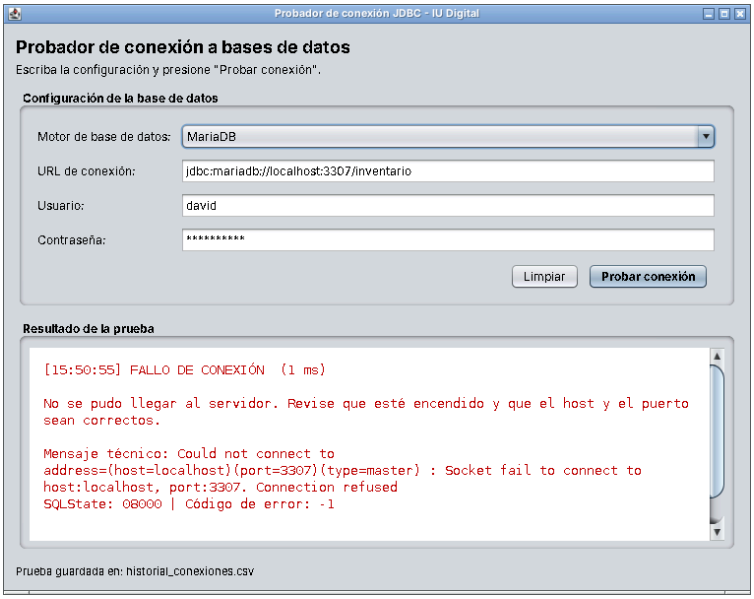
Base de datos que no existe (SQLException)



Nota. Error 1049: el servidor responde, pero la base “tienda” no existe. Captura de la aplicación en ejecución.

Figura 6

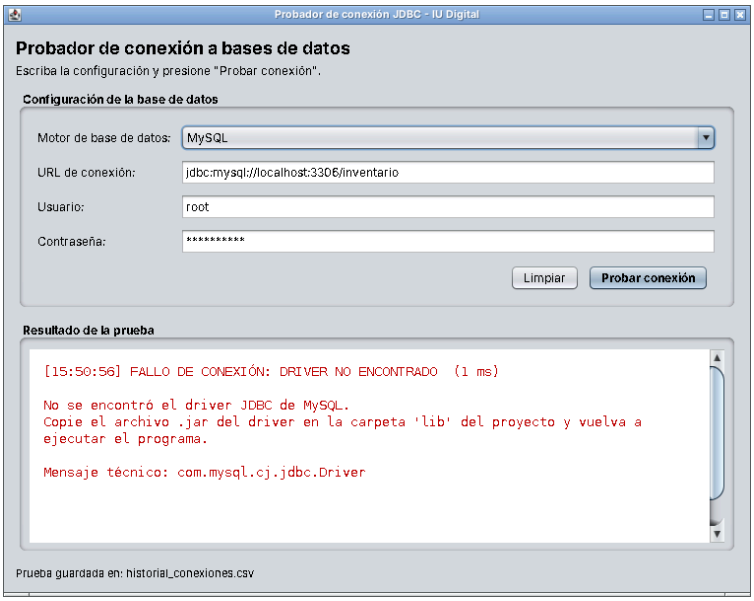
Servidor apagado o puerto equivocado (SQLException)



Nota. Se usó el puerto 3307, donde no hay ningún servidor. Captura de la aplicación en ejecución.

Figura 7

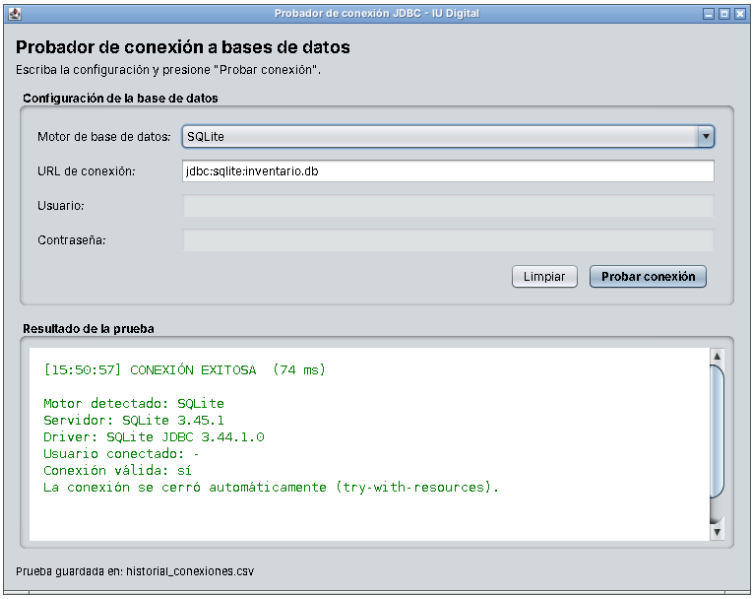
Driver de MySQL no encontrado (ClassNotFoundException)



Nota. Se probó sin el archivo mysql-connector-j en la carpeta lib. Captura de la aplicación en ejecución.

Figura 8

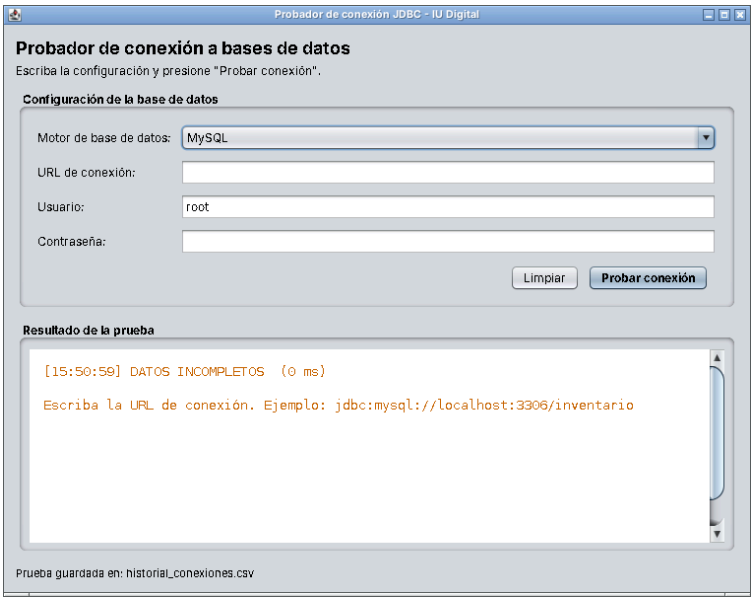
Conexión exitosa con SQLite



Nota. Con SQLite los campos de usuario y contraseña se desactivan. Captura de la aplicación en ejecución.

Figura 9

Validación de datos: URL vacía (mensaje naranja)



Nota. El programa avisa antes de intentar conectarse. Captura de la aplicación en ejecución.

Tabla 4

Contenido del archivo historial_conexiones.csv después de las pruebas

Hora	URL	Usuario	Resultado	Tipo de error	ms
15:50:52	`jdbc:mariadb://localhost:3306/inventario`	david	EXITOSA	NINGUNO	91
15:50:53	`jdbc:mariadb://localhost:3306/inventario`	david	FALLIDA	FALLO_CONEXION	12
15:50:54	`jdbc:mariadb://localhost:3306/tienda`	admin	FALLIDA	FALLO_CONEXION	2
15:50:55	`jdbc:mariadb://localhost:3307/inventario`	david	FALLIDA	FALLO_CONEXION	1
15:50:56	`jdbc:mysql://localhost:3306/inventario`	root	FALLIDA	DRIVER_NO_ENCONTRADO	1
15:50:57	`jdbc:sqlite:inventario.db`		EXITOSA	NINGUNO	74
15:50:59	`(vacía)`	root	FALLIDA	DATOS_INVALIDOS	0

Nota. Se omite la columna “mensaje” por espacio. El archivo completo está en la carpeta docs del proyecto. La prueba 01 no aparece porque solo es la ventana inicial.

Repositorio y Archivo ZIP

El proyecto se publicó en un repositorio público de GitHub, en la rama principal `main`:

https://github.com/davidezapatar/ConexionTester

El archivo ZIP de la entrega contiene:

- Este informe en PDF.
- La carpeta `src` con el código fuente en Java (6 clases en 3 paquetes).
- `docs/codigo_fuente.csv`: el código fuente completo en formato CSV, con una fila por cada línea de código y las columnas archivo, paquete, clase, número de línea y código.
- `docs/historial_conexiones_pruebas.csv`: el historial de las pruebas de la Tabla 4.
- `ejecutar.bat` (Windows) y `ejecutar.sh` (Linux/Mac) para compilar y abrir la aplicación con un solo paso, el `README.md` y la carpeta `lib` con las instrucciones para descargar el driver.

El driver de MySQL no se incluye dentro del proyecto porque se descarga gratis desde la página oficial de MySQL (https://dev.mysql.com/downloads/connector/j/, opción *Platform Independent*). Basta con copiar el archivo `mysql-connector-j-X.X.X.jar` en la carpeta `lib`. Si no se copia, la aplicación lo detecta y muestra el error de la Figura 7, en lugar de cerrarse.

Conclusiones

- Se cumplió el objetivo general: la aplicación permite escribir la configuración de cualquier base de datos, probar la conexión y ver claramente si funcionó (verde) o por qué falló (rojo), con el mensaje técnico del error.
- Swing permitió construir una interfaz completa con los componentes pedidos. Combinar `BorderLayout` y `GridLayout` dejó el formulario ordenado y alineado, y el `SwingWorker` evita que la ventana se congele mientras espera al servidor.
- JDBC hace posible que el mismo código funcione con MySQL, MariaDB, SQLite y otros motores: lo único que cambia es la URL y el driver. `DriverManager.getConnection()` recibe la configuración que escribe el usuario, sin datos fijos en el código.
- El *try-with-resources* garantiza que la conexión siempre se cierre, salga bien o mal, sin depender de que el programador se acuerde de cerrarla.
- Atrapar `ClassNotFoundException` y `SQLException` por separado y traducir sus códigos permitió dar mensajes útiles para cinco tipos de error diferentes, en lugar de un error genérico o de que el programa se cierre.
- Separar la ventana (`ConexionTester`) de la lógica (`ValidadorConexion` e `HistorialCsv`) hace que el código sea más ordenado, fácil de leer y de reutilizar, como piden los principios de la programación orientada a objetos.

Enlaces del Repositorio

Tabla 5

Repositorio de cada integrante del grupo

Integrante	Usuario de GitHub	Enlace del repositorio
David Esteban Zapata Rodríguez	davidezapatar	https://github.com/davidezapatar/ConexionTester

Nota. El trabajo se realizó de forma individual en el Grupo 10.

Referencias

OpenJDK. (2021). *JEP 395: Records*. https://openjdk.org/jeps/395

Oracle. (s.f.-a). *JDBC basics*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

Oracle. (s.f.-b). *Creating a GUI with Swing*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/

Oracle. (s.f.-c). *A visual guide to layout managers*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html

Oracle. (s.f.-d). *Worker threads and SwingWorker*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/concurrency/worker.html

Oracle. (s.f.-e). *The try-with-resources statement*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html

Oracle. (s.f.-f). *Handling SQLExceptions*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/sqlexception.html

Anexo. Código Fuente Completo

Código fuente completo del proyecto, igual al que está en el repositorio.

src/conexiontester/ConexionTester.java

```
package conexiontester;

import conexiontester.modelo.ConfiguracionConexion;
import conexiontester.modelo.ResultadoPrueba;
import conexiontester.modelo.TipoBaseDatos;
import conexiontester.servicio.HistorialCsv;
import conexiontester.servicio.ValidadorConexion;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;
import javax.swing.SwingWorker;
import javax.swing.UIManager;
import javax.swing.WindowConstants;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridLayout;
import java.io.IOException;
import java.nio.file.Path;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.ExecutionException;

/**
 * Ventana principal: permite escribir la configuracion de una base de datos
 * y probar si la conexion funciona.
 *
 * Esta clase solo se encarga de la interfaz grafica (Swing). La conexion la
 * hace ValidadorConexion y el historial lo guarda HistorialCsv.
 */
public final class ConexionTester extends JFrame {

    private static final long serialVersionUID = 1L;

    private static final Color VERDE = new Color(0, 128, 0);
    private static final Color ROJO = new Color(190, 0, 0);
    private static final Color NARANJA = new Color(200, 100, 0);
    private static final Font FUENTE_RESULTADO = new Font(Font.MONOSPACED, Font.PLAIN, 13);
    private static final DateTimeFormatter FORMATO_HORA = DateTimeFormatter.ofPattern("HH:mm:ss");

    // Componentes de la interfaz
    private final JComboBox<TipoBaseDatos> comboMotor = new JComboBox<>(TipoBaseDatos.values());
    private final JTextField campoUrl = new JTextField(35);
    private final JTextField campoUsuario = new JTextField(20);
    private final JPasswordField campoContrasena = new JPasswordField(20);
    private final JButton botonProbar = new JButton("Probar conexión");
    private final JButton botonLimpiar = new JButton("Limpiar");
    private final JTextArea areaResultado = new JTextArea(12, 60);
    private final JLabel etiquetaEstado = new JLabel();

    // Logica (separada de la interfaz)
    private final transient ValidadorConexion validador;
    private final transient HistorialCsv historial;

    public ConexionTester(ValidadorConexion validador, HistorialCsv historial) {
        super("Probador de conexión JDBC - IU Digital");
        this.validador = validador;
        this.historial = historial;
        construirInterfaz();
```

```
        registrarEventos();
        seleccionarMotor(TipoBaseDatos.MYSQL);
    }

    // ------------------------------------------------------------------
    // Construccion de la interfaz
    // ------------------------------------------------------------------

    private void construirInterfaz() {
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);

        JPanel panelPrincipal = new JPanel(new BorderLayout(10, 10));
        panelPrincipal.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
        panelPrincipal.add(crearPanelEncabezado(), BorderLayout.NORTH);
        panelPrincipal.add(crearPanelCentral(), BorderLayout.CENTER);
        panelPrincipal.add(crearBarraEstado(), BorderLayout.SOUTH);

        setContentPane(panelPrincipal);
        pack();
        setMinimumSize(new Dimension(640, 520));
        setLocationRelativeTo(null);
    }

    private JPanel crearPanelEncabezado() {
        JPanel panel = new JPanel(new GridLayout(2, 1));
        JLabel titulo = new JLabel("Probador de conexión a bases de datos");
        titulo.setFont(titulo.getFont().deriveFont(Font.BOLD, 18f));
        JLabel subtitulo = new JLabel("Escriba la configuración y presione \"Probar conexión\".");
        panel.add(titulo);
        panel.add(subtitulo);
        return panel;
    }

    /** Formulario arriba y resultado abajo. */
    private JPanel crearPanelCentral() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.add(crearPanelFormulario(), BorderLayout.NORTH);
        panel.add(crearPanelResultado(), BorderLayout.CENTER);
        return panel;
    }

    /**
     * Formulario: una columna de etiquetas (WEST) y una de campos (CENTER),
     * cada una con GridLayout para que queden alineadas fila por fila.
     */
    private JPanel crearPanelFormulario() {
        JPanel etiquetas = new JPanel(new GridLayout(4, 1, 5, 8));
        JPanel campos = new JPanel(new GridLayout(4, 1, 5, 8));

        agregarFila(etiquetas, campos, "Motor de base de datos:", comboMotor);
        agregarFila(etiquetas, campos, "URL de conexión:", campoUrl);
        agregarFila(etiquetas, campos, "Usuario:", campoUsuario);
        agregarFila(etiquetas, campos, "Contraseña:", campoContrasena);

        JPanel botones = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
        botonProbar.setFont(botonProbar.getFont().deriveFont(Font.BOLD));
        botones.add(botonLimpiar);
        botones.add(botonProbar);

        JPanel formulario = new JPanel(new BorderLayout(10, 10));
        formulario.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createTitledBorder("Configuración de la base de datos"),
                BorderFactory.createEmptyBorder(6, 8, 8, 8)));
        formulario.add(etiquetas, BorderLayout.WEST);
        formulario.add(campos, BorderLayout.CENTER);
        formulario.add(botones, BorderLayout.SOUTH);

        // Nombres para poder ubicar los componentes en las pruebas automaticas
        comboMotor.setName("comboMotor");
        campoUrl.setName("campoUrl");
        campoUsuario.setName("campoUsuario");
        campoContrasena.setName("campoContrasena");
        botonProbar.setName("botonProbar");
        areaResultado.setName("areaResultado");
        return formulario;
    }

    private void agregarFila(JPanel etiquetas, JPanel campos, String texto, java.awt.Component campo) {
        JLabel etiqueta = new JLabel(texto);
        etiqueta.setLabelFor(campo);
```

```
        etiquetas.add(etiqueta);
        campos.add(campo);
    }

    private JScrollPane crearPanelResultado() {
        areaResultado.setEditable(false);
        areaResultado.setLineWrap(true);
        areaResultado.setWrapStyleWord(true);
        areaResultado.setFont(FUENTE_RESULTADO);
        areaResultado.setMargin(new java.awt.Insets(8, 8, 8, 8));

        JScrollPane scroll = new JScrollPane(areaResultado);
        scroll.setBorder(BorderFactory.createTitledBorder("Resultado de la prueba"));
        return scroll;
    }

    private JLabel crearBarraEstado() {
        etiquetaEstado.setText("Historial de pruebas: " + historial.getArchivo().getFileName());
        etiquetaEstado.setFont(etiquetaEstado.getFont().deriveFont(11f));
        return etiquetaEstado;
    }

    // ------------------------------------------------------------------
    // Eventos
    // ------------------------------------------------------------------

    private void registrarEventos() {
        // ActionListener del boton "Probar conexion"
        botonProbar.addActionListener(evento -> probarConexion());
        botonLimpiar.addActionListener(evento -> limpiarResultado());
        comboMotor.addActionListener(evento -> seleccionarMotor((TipoBaseDatos) comboMotor.getSelectedItem()));
        // Presionar Enter en cualquier campo tambien prueba la conexion
        getRootPane().setDefaultButton(botonProbar);
    }

    /** Cambia la URL de ejemplo y activa o desactiva usuario y contrasena (SQLite no los usa). */
    private void seleccionarMotor(TipoBaseDatos tipo) {
        if (comboMotor.getSelectedItem() != tipo) {
            comboMotor.setSelectedItem(tipo);
        }
        campoUrl.setText(tipo.getUrlEjemplo());
        boolean usaCredenciales = tipo.necesitaCredenciales();
        campoUsuario.setEnabled(usaCredenciales);
        campoContrasena.setEnabled(usaCredenciales);
        if (!usaCredenciales) {
            campoUsuario.setText("");
            campoContrasena.setText("");
        }
    }

    /** Lee los campos de texto y arma la configuracion (nada esta escrito fijo en el codigo). */
    private ConfiguracionConexion leerConfiguracion() {
        return new ConfiguracionConexion(
                campoUrl.getText(),
                campoUsuario.getText(),
                new String(campoContrasena.getPassword()));
    }

    /**
     * Ejecuta la prueba en segundo plano con SwingWorker, para que la ventana
     * no se congele mientras espera al servidor.
     */
    private void probarConexion() {
        ConfiguracionConexion configuracion = leerConfiguracion();
        botonProbar.setEnabled(false);
        setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));
        areaResultado.setForeground(Color.DARK_GRAY);
        areaResultado.setText("Probando conexión con " + configuracion.url() + " ...");

        new SwingWorker<ResultadoPrueba, Void>() {
            @Override
            protected ResultadoPrueba doInBackground() {
                return validador.probarConexion(configuracion);
            }

            @Override
            protected void done() {
                try {
                    ResultadoPrueba resultado = get();
                    mostrarResultado(resultado);
```

```
                    guardarEnHistorial(configuracion, resultado);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } catch (ExecutionException e) {
                    mostrarErrorInesperado(e.getCause());
                } finally {
                    botonProbar.setEnabled(true);
                    setCursor(Cursor.getDefaultCursor());
                }
            }
        }.execute();
    }

    /** Verde si la conexion funciono, rojo si fallo, naranja si faltan datos. */
    private void mostrarResultado(ResultadoPrueba resultado) {
        Color color = switch (resultado.tipoError()) {
            case NINGUNO -> VERDE;
            case DATOS_INVALIDOS -> NARANJA;
            case DRIVER_NO_ENCONTRADO, FALLO_CONEXION -> ROJO;
        };
        areaResultado.setForeground(color);
        areaResultado.setText("[" + resultado.fechaHora().format(FORMATO_HORA) + "] "
                + resultado.titulo() + "  (" + resultado.milisegundos() + " ms)\n\n"
                + resultado.detalle());
        areaResultado.setCaretPosition(0);
    }

    private void guardarEnHistorial(ConfiguracionConexion configuracion, ResultadoPrueba resultado) {
        try {
            historial.registrar(configuracion, resultado);
            etiquetaEstado.setText("Prueba guardada en: " + historial.getArchivo().getFileName());
        } catch (IOException e) {
            etiquetaEstado.setText("No se pudo guardar el historial: " + e.getMessage());
        }
    }

    private void mostrarErrorInesperado(Throwable causa) {
        areaResultado.setForeground(ROJO);
        areaResultado.setText("ERROR INESPERADO\n\n" + causa);
    }

    private void limpiarResultado() {
        areaResultado.setText("");
        campoContrasena.setText("");
    }

    // ------------------------------------------------------------------
    // Punto de entrada
    // ------------------------------------------------------------------

    public static void main(String[] args) {
        usarAparienciaNimbus();
        SwingUtilities.invokeLater(() -> {
            ConexionTester ventana = new ConexionTester(
                    new ValidadorConexion(),
                    new HistorialCsv(Path.of("historial_conexiones.csv")));
            ventana.setVisible(true);
        });
    }

    /** Usa el estilo "Nimbus" que trae Java; si no esta, deja el estilo por defecto. */
    private static void usarAparienciaNimbus() {
        for (UIManager.LookAndFeelInfo estilo : UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(estilo.getName())) {
                try {
                    UIManager.setLookAndFeel(estilo.getClassName());
                } catch (ReflectiveOperationException | javax.swing.UnsupportedLookAndFeelException e) {
                    // Se queda con el estilo por defecto
                }
                return;
            }
        }
    }
}
```

src/conexiontester/modelo/ConfiguracionConexion.java

```
package conexiontester.modelo;

/**
 * Datos que escribe el usuario en la ventana para conectarse.
 * Es un record: sus valores no se pueden cambiar despues de creado.
 *
 * @param url        URL JDBC, por ejemplo jdbc:mysql://localhost:3306/inventario
 * @param usuario    usuario de la base de datos (vacio en SQLite)
 * @param contrasena contrasena del usuario (vacia en SQLite)
 */
public record ConfiguracionConexion(String url, String usuario, String contrasena) {

    public ConfiguracionConexion {
        url = url == null ? "" : url.trim();
        usuario = usuario == null ? "" : usuario.trim();
        contrasena = contrasena == null ? "" : contrasena;
    }

    public boolean tieneUsuario() {
        return !usuario.isEmpty();
    }

    /** Evita que la contrasena aparezca si alguien imprime el objeto. */
    @Override
    public String toString() {
        return "ConfiguracionConexion[url=" + url + ", usuario=" + usuario + ", contrasena=****]";
    }
}
```

src/conexiontester/modelo/ResultadoPrueba.java

```
package conexiontester.modelo;

import java.time.LocalDateTime;

/**
 * Resultado de una prueba de conexion.
 *
 * @param exitosa      true si la conexion se pudo abrir
 * @param tipoError    NINGUNO, DATOS_INVALIDOS, DRIVER_NO_ENCONTRADO o FALLO_CONEXION
 * @param titulo       mensaje corto ("Conexion exitosa", "Fallo de conexion"...)
 * @param detalle      explicacion y mensaje tecnico del error (e.getMessage())
 * @param milisegundos tiempo que tardo la prueba
 * @param fechaHora    momento en que se hizo la prueba
 */
public record ResultadoPrueba(boolean exitosa,
                              TipoError tipoError,
                              String titulo,
                              String detalle,
                              long milisegundos,
                              LocalDateTime fechaHora) {

    /** Tipos de resultado posibles. */
    public enum TipoError {
        NINGUNO,
        DATOS_INVALIDOS,
        DRIVER_NO_ENCONTRADO,
        FALLO_CONEXION
    }

    public static ResultadoPrueba exito(String detalle, long milisegundos) {
        return new ResultadoPrueba(true, TipoError.NINGUNO, "CONEXIÓN EXITOSA", detalle,
                milisegundos, LocalDateTime.now());
    }

    public static ResultadoPrueba error(TipoError tipo, String titulo, String detalle, long milisegundos) {
        return new ResultadoPrueba(false, tipo, titulo, detalle, milisegundos, LocalDateTime.now());
    }
}
```

src/conexiontester/modelo/TipoBaseDatos.java

```
package conexiontester.modelo;
```

```
import java.util.Arrays;
import java.util.Optional;

/**
 * Motores de bases de datos que reconoce la aplicacion.
 * Cada motor sabe el inicio de su URL JDBC, el nombre de la clase de su driver
 * y un ejemplo de URL para ayudar al usuario.
 */
public enum TipoBaseDatos {

    MYSQL("MySQL", "jdbc:mysql:", "com.mysql.cj.jdbc.Driver",
            "jdbc:mysql://localhost:3306/inventario"),
    MARIADB("MariaDB", "jdbc:mariadb:", "org.mariadb.jdbc.Driver",
            "jdbc:mariadb://localhost:3306/inventario"),
    POSTGRESQL("PostgreSQL", "jdbc:postgresql:", "org.postgresql.Driver",
            "jdbc:postgresql://localhost:5432/inventario"),
    SQL_SERVER("SQL Server", "jdbc:sqlserver:", "com.microsoft.sqlserver.jdbc.SQLServerDriver",
            "jdbc:sqlserver://localhost:1433;databaseName=inventario;encrypt=false"),
    SQLITE("SQLite", "jdbc:sqlite:", "org.sqlite.JDBC",
            "jdbc:sqlite:inventario.db");

    private final String nombre;
    private final String prefijoUrl;
    private final String claseDriver;
    private final String urlEjemplo;

    TipoBaseDatos(String nombre, String prefijoUrl, String claseDriver, String urlEjemplo) {
        this.nombre = nombre;
        this.prefijoUrl = prefijoUrl;
        this.claseDriver = claseDriver;
        this.urlEjemplo = urlEjemplo;
    }

    public String getNombre() {
        return nombre;
    }

    public String getClaseDriver() {
        return claseDriver;
    }

    public String getUrlEjemplo() {
        return urlEjemplo;
    }

    /** SQLite trabaja con un archivo y no necesita usuario ni contrasena. */
    public boolean necesitaCredenciales() {
        return this != SQLITE;
    }

    /**
     * Busca el motor segun el inicio de la URL.
     * Ejemplo: "jdbc:mysql://..." devuelve MYSQL.
     */
    public static Optional<TipoBaseDatos> desdeUrl(String url) {
        String urlMinuscula = url.trim().toLowerCase();
        return Arrays.stream(values())
                .filter(tipo -> urlMinuscula.startsWith(tipo.prefijoUrl))
                .findFirst();
    }

    @Override
    public String toString() {
        return nombre;
    }
}
```

src/conexiontester/servicio/ValidadorConexion.java

```
package conexiontester.servicio;

import conexiontester.modelo.ConfiguracionConexion;
import conexiontester.modelo.ResultadoPrueba;
import conexiontester.modelo.ResultadoPrueba.TipoError;
import conexiontester.modelo.TipoBaseDatos;

import java.sql.Connection;
```

```
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Optional;

/**
 * Logica de conexion a la base de datos, separada de la interfaz grafica.
 *
 * Esta clase no sabe nada de Swing: recibe una ConfiguracionConexion y
 * devuelve un ResultadoPrueba. Por eso se puede usar desde la ventana,
 * desde la consola o desde una prueba.
 */
public class ValidadorConexion {

    /** Segundos maximos que se espera al servidor antes de rendirse. */
    private static final int TIEMPO_ESPERA_SEGUNDOS = 5;

    /**
     * Intenta abrir una conexion con los datos recibidos y la cierra enseguida.
     *
     * @param configuracion URL, usuario y contrasena escritos por el usuario
     * @return el resultado de la prueba (exito o el tipo de error con su mensaje)
     */
    public ResultadoPrueba probarConexion(ConfiguracionConexion configuracion) {
        long inicio = System.currentTimeMillis();

        // 1. Revisar que los datos tengan sentido antes de intentar conectar
        Optional<String> errorDatos = validarDatos(configuracion);
        if (errorDatos.isPresent()) {
            return ResultadoPrueba.error(TipoError.DATOS_INVALIDOS, "DATOS INCOMPLETOS",
                    errorDatos.get(), calcularTiempo(inicio));
        }

        Optional<TipoBaseDatos> tipo = TipoBaseDatos.desdeUrl(configuracion.url());

        try {
            // 2. Cargar el driver JDBC. Si el .jar no esta en la carpeta lib,
            //    Java lanza ClassNotFoundException.
            if (tipo.isPresent()) {
                Class.forName(tipo.get().getClaseDriver());
            }

            // 3. Abrir la conexion con try-with-resources: Java la cierra
            //    automaticamente al salir del bloque, salga bien o salga mal.
            DriverManager.setLoginTimeout(TIEMPO_ESPERA_SEGUNDOS);
            try (Connection conexion = abrirConexion(configuracion)) {
                String detalle = describirConexion(conexion, tipo);
                return ResultadoPrueba.exito(detalle, calcularTiempo(inicio));
            }

        } catch (ClassNotFoundException e) {
            String motor = tipo.map(TipoBaseDatos::getNombre).orElse("desconocido");
            String detalle = "No se encontró el driver JDBC de " + motor + ".\n"
                    + "Copie el archivo .jar del driver en la carpeta 'lib' del proyecto y vuelva a ejecutar el
programa.\n\n"
                    + "Mensaje técnico: " + e.getMessage();
            return ResultadoPrueba.error(TipoError.DRIVER_NO_ENCONTRADO, "FALLO DE CONEXIÓN: DRIVER NO ENCONTRADO",
                    detalle, calcularTiempo(inicio));

        } catch (SQLException e) {
            String detalle = explicarError(e) + "\n\n"
                    + "Mensaje técnico: " + e.getMessage() + "\n"
                    + "SQLState: " + e.getSQLState() + " | Código de error: " + e.getErrorCode();
            return ResultadoPrueba.error(TipoError.FALLO_CONEXION, "FALLO DE CONEXIÓN",
                    detalle, calcularTiempo(inicio));
        }
    }

    /** Revisa los datos escritos. Devuelve el problema encontrado, o vacio si todo esta bien. */
    Optional<String> validarDatos(ConfiguracionConexion configuracion) {
        if (configuracion.url().isEmpty()) {
            return Optional.of("Escriba la URL de conexión. Ejemplo: jdbc:mysql://localhost:3306/inventario");
        }
        if (!configuracion.url().toLowerCase().startsWith("jdbc:")) {
            return Optional.of("La URL debe empezar por 'jdbc:'. Ejemplo: jdbc:mysql://localhost:3306/inventario");
        }
        return Optional.empty();
    }

    /** SQLite no usa usuario: en ese caso se llama a getConnection solo con la URL. */
```

```
    private Connection abrirConexion(ConfiguracionConexion configuracion) throws SQLException {
        if (configuracion.tieneUsuario()) {
            return DriverManager.getConnection(configuracion.url(), configuracion.usuario(),
                    configuracion.contrasena());
        }
        return DriverManager.getConnection(configuracion.url());
    }

    /** Arma el texto con la informacion del servidor al que se conecto. */
    private String describirConexion(Connection conexion, Optional<TipoBaseDatos> tipo) throws SQLException {
        DatabaseMetaData datos = conexion.getMetaData();
        return "Motor detectado: " + tipo.map(TipoBaseDatos::getNombre).orElse("otro") + "\n"
                + "Servidor: " + datos.getDatabaseProductName() + " " + datos.getDatabaseProductVersion() + "\n"
                + "Driver: " + datos.getDriverName() + " " + datos.getDriverVersion() + "\n"
                + "Usuario conectado: " + valorOGuion(datos.getUserName()) + "\n"
                + "Conexión válida: " + (conexion.isValid(TIEMPO_ESPERA_SEGUNDOS) ? "sí" : "no") + "\n"
                + "La conexión se cerró automáticamente (try-with-resources).";
    }

    /** Traduce los errores mas comunes a un mensaje facil de entender. */
    String explicarError(SQLException e) {
        String estado = e.getSQLState() == null ? "" : e.getSQLState();
        String mensaje = e.getMessage() == null ? "" : e.getMessage().toLowerCase();

        if (mensaje.contains("no suitable driver")) {
            return "La URL no corresponde a ningún driver cargado. Revise que esté bien escrita.";
        }
        if (e.getErrorCode() == 1049 || mensaje.contains("unknown database")) {
            return "El servidor respondió, pero la base de datos no existe.";
        }
        if (e.getErrorCode() == 1044) {
            return "El usuario y la contraseña son correctos, pero ese usuario no tiene permiso "
                    + "sobre esa base de datos (o la base de datos no existe).";
        }
        if (estado.startsWith("28") || e.getErrorCode() == 1045 || mensaje.contains("access denied")) {
            return "El usuario o la contraseña son incorrectos.";
        }
        if (estado.startsWith("08") || mensaje.contains("communications link failure")
                || mensaje.contains("connection refused") || mensaje.contains("could not connect")) {
            return "No se pudo llegar al servidor. Revise que esté encendido y que el host y el puerto sean
correctos.";
        }
        return "La base de datos rechazó la conexión.";
    }

    private static String valorOGuion(String valor) {
        return valor == null || valor.isBlank() ? "-" : valor;
    }

    private static long calcularTiempo(long inicio) {
        return System.currentTimeMillis() - inicio;
    }
}
```

src/conexiontester/servicio/HistorialCsv.java

```
package conexiontester.servicio;

import conexiontester.modelo.ConfiguracionConexion;
import conexiontester.modelo.ResultadoPrueba;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.time.format.DateTimeFormatter;
import java.util.List;

/**
 * Guarda cada prueba de conexion en un archivo CSV (historial_conexiones.csv).
 * La contrasena NUNCA se guarda.
 */
public class HistorialCsv {

    static final String ENCABEZADO = "fecha_hora;url;usuario;resultado;tipo_error;tiempo_ms;mensaje";
    private static final DateTimeFormatter FORMATO_FECHA = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
```

```
    private final Path archivo;

    public HistorialCsv(Path archivo) {
        this.archivo = archivo;
    }

    public Path getArchivo() {
        return archivo;
    }

    /**
     * Agrega una fila al CSV. Si el archivo no existe, primero escribe el encabezado.
     *
     * @throws IOException si no se puede escribir el archivo
     */
    public void registrar(ConfiguracionConexion configuracion, ResultadoPrueba resultado) throws IOException {
        if (Files.notExists(archivo)) {
            // El BOM al inicio hace que Excel muestre bien las tildes
            Files.writeString(archivo, "" + ENCABEZADO + System.lineSeparator(), StandardCharsets.UTF_8);
        }
        String fila = String.join(";",
                resultado.fechaHora().format(FORMATO_FECHA),
                limpiar(configuracion.url()),
                limpiar(configuracion.usuario()),
                resultado.exitosa() ? "EXITOSA" : "FALLIDA",
                resultado.tipoError().name(),
                String.valueOf(resultado.milisegundos()),
                limpiar(resultado.detalle().replace("\n\n", "\n").replace("\n", " | ")));
        Files.write(archivo, List.of(fila), StandardCharsets.UTF_8, StandardOpenOption.APPEND);
    }

    /** Pone el texto entre comillas si tiene punto y coma, comillas o saltos de linea. */
    static String limpiar(String texto) {
        String sinSaltos = texto.replace("\r", " ").replace("\n", " ");
        if (sinSaltos.contains(";") || sinSaltos.contains("\"")) {
            return "\"" + sinSaltos.replace("\"", "\"\"") + "\"";
        }
        return sinSaltos;
    }
}
```